Two Pointers

125. Valid Palindrome

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` *if it is a* ***palindrome****, or* `false` *otherwise*.

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

```
class Solution {
public:
    bool isAlphaNum(char ch){
        if((ch >= '0' && ch <= '9') ||
            (tolower(ch) >= 'a' && tolower(ch) <= 'z')){
                return true;
            }
            return false;
    }
    bool isPalindrome(string s) {
        int st =0  , end = s.length()-1;

        while(st < end){
            if(!isAlphaNum(s[st])){
                st++ ; continue;
            }
            if(!isAlphaNum(s[end])){
                end-- ; continue;
            }
```

```
        if(tolower(s[st]) != tolower(s[end])){
            return false;
        }
        st++; end--;
    }
    return true;
  }
};
```

Two Pointers

125. Valid Palindrome
Is Subsequence
Two Sum II - Input Array is Sorted
Container with most water
3 Sum

⇒ **125. Valid Palindrome**

consider
alphaNumeric ⇒ a-z, A-Z, 0-9
not consider special characters

eg "A c 3 ? e 3 c & a"

= Two Pointer approach
1st convert to lowercase = a c 3 ? e 3 c & a
start end

a == a ⇒ start++, end--

⇒ a c 3 ? e 3 c & a ⇒ end-- ⇒ a c 3 ? e 3 c & a
st end start end

c == c

⇒ a c 3 ? e 3 c & a ⇒ 3 == 3 ⇒ a c 3 ? e 3 c & a
start end start end

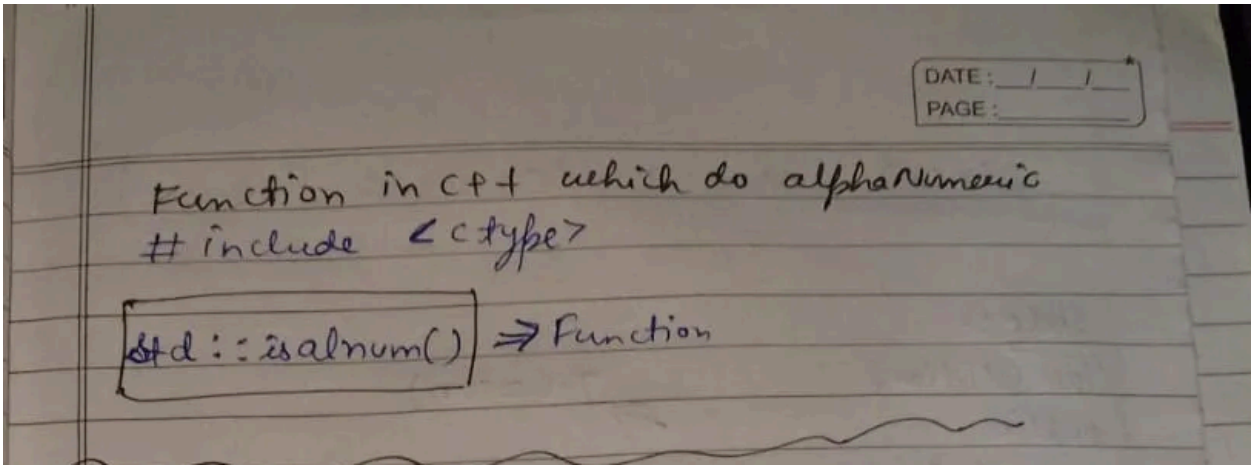
⇒ start++
a c 3 ? e 3 c & a
st end

DATE: __/__/__
PAGE: ____

Code

```
class Solution {
public:
    bool isAlphaNum( char ch) {
        if( (ch >= '0' && ch <= '9') ||
            (tolower(ch) >= 'a' &&
            tolower(ch) <= 'z')) {
O(1) →          return true;
        }
        return false;
    }

    bool isPalindrome (string s) {
        int st=0 , end = s.length() -1;
        while ( st < end ) {
O(n) →      if( !isAlphaNum(s[st])) {
                st++; continue;
            }
            if (!isAlphaNum ( s[end])) {
                end--; continue;
            }
            if (tolower(s[st]) != tolower(s[end])) {
                return false;
            }
            st++;   end--;
        }
        return true;
    }
};
```

T.C = O(n)



392. Is Subsequence

Given two strings `s` and `t`, return `true` *if* `s` *is a* ***subsequence*** *of* `t`, *or* `false` *otherwise*.

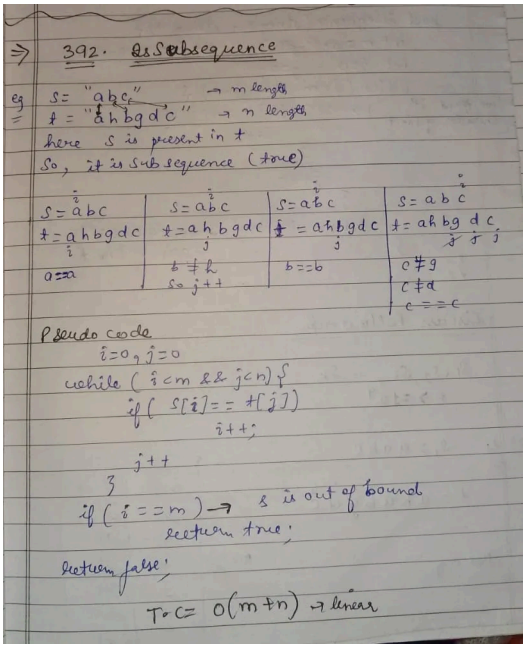
A **subsequence** of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., `"ace"` is a subsequence of `"abcde"` while `"aec"` is not).

Example 1:

```
Input: s = "abc", t = "ahbgdc"
Output: true
```

```
class Solution {
public:
   bool isSubsequence(string s, string t) {
      int m  = s.length();
      int n  = t.length();
      int i = 0, j = 0;

      while(i < m && j<n) {
         if(s[i] == t[j])
            i++;
         j++;
```

```
        }

        return i == m;
    }
};
```



⇒ 392. Is Subsequence

eg: s = "abc" → m length
t = "ahbgdc" → n length
here s is present in t
So, it is subsequence (true)

s = abc	s = abc	s = abc	s = abc
t = ahbgdc	t = ahbgdc	t = ahbgdc	t = ahbgdc
a == a	b ≠ h So j++	b == b	c ≠ g c ≠ d c == c

Pseudo code

```
i=0, j=0
while ( i<m && j<n) {
    if( s[i] == t[j])
        i++;
    j++
}
if ( i == m) → s is out of bound
    return true;
return false;
```

T.C = O(m+n) → linear

Code :-

```
class Solution {
public:
    bool isSequence(string s, string t) {
        int m = s.length();   int n = t.length();
        int i=0 , j=0;
        while (i<m && j<n) {
            if( s[i] == t[j]) {
                i++;
            }
            j++;
        }
        return i==m;
    }
};
```

T.C = O(n)

we found ith character of (s) in (t)

return i==m; → we found all chars of (s) in (t)

Discuss follow-up

$S_1, S_2, S_3, \ldots S_k$

$k >= 10^9$

⇒ eg $S_1 = abc$

$t = \overset{0}{a}\overset{1}{h}\overset{2}{a}\overset{3}{b}\overset{4}{g}\overset{5}{d}\overset{6}{c}$

use map

map ⇒

(t) char	indices
a	{0, 2}
h	{1}
b	{3}
g	{4}
d	{5}
c	{6}

167. Two Sum II - Input Array Is Sorted

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int i = 0, j = numbers.size()-1;

        while(i < j) {
            if(numbers[i]+numbers[j] == target) {
                return {i+1, j+1};
            }

            if(numbers[i]+numbers[j] > target)
                j--;
            else
                i++;
        }
        return {};
    }
};
```

11. Container With Most Water

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int n = height.size();
        int i = 0, j = n-1;

        int maxwater = 0;
        while(i<j) {

            //start from the smallest one and calculate water
            int h = min(height[i], height[j]);
```

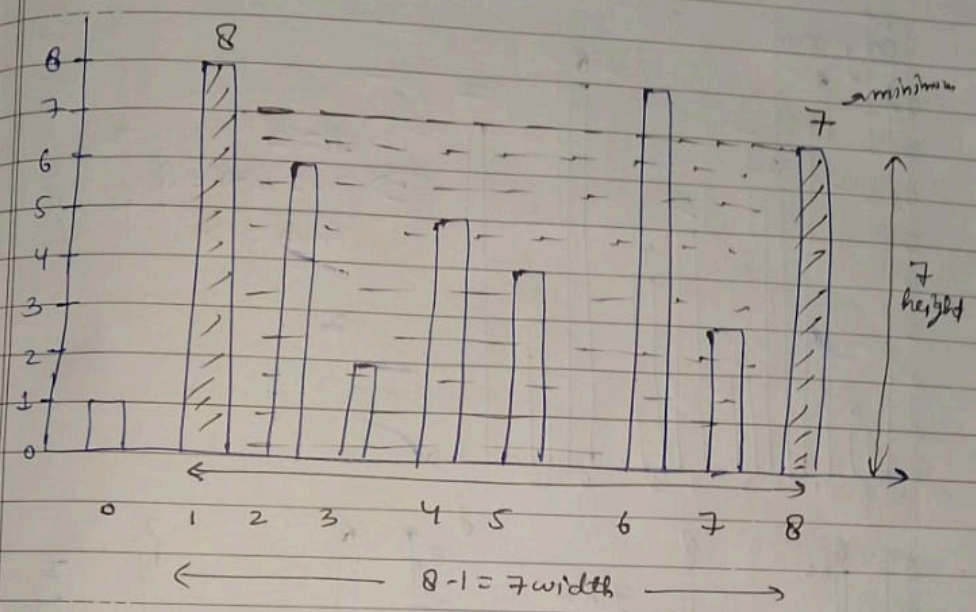
```
        int w = j-i;
        int area = h*w;
        maxwater = max(maxwater, area);

        if(height[i] < height[j])
            i++;
        else
            j--;
    }

    return maxwater;
  }
};
```

DATE: __/__/__
PAGE: __

Container with most water

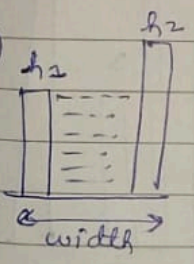
i/p height = [1, 8, 6, 2, 5, 4, 8, 3, 7]

o/p = 49



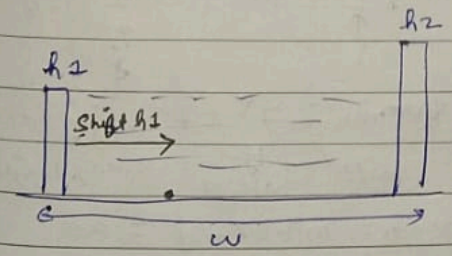
Area = 7X7 (width × height)
= 49 → maximum area

1

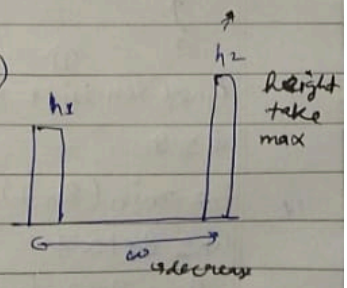


height = min(h1, h2);

2



A(area)



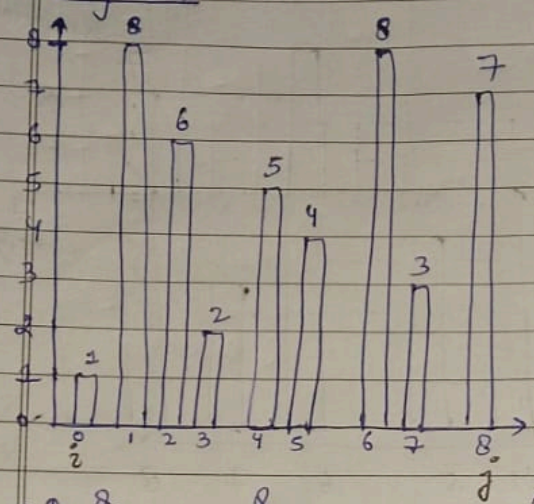
if we shift h1 then width is decrease but we need to choose large height for max area

DATE: __/__/__
PAGE: ____

$\boxed{\text{Area} = w \times h}$

if width(w) decrease then we need to choose large height(↑) to maximize the area

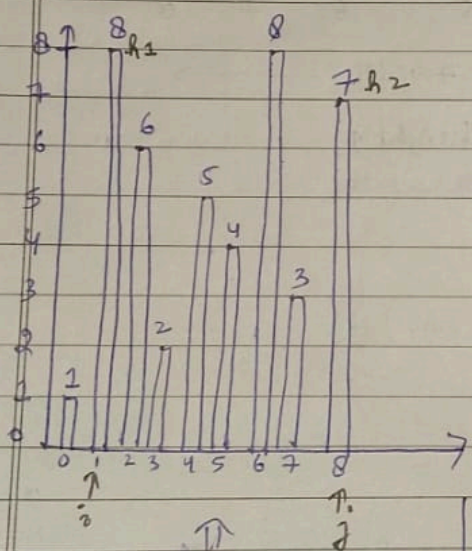
Dry run

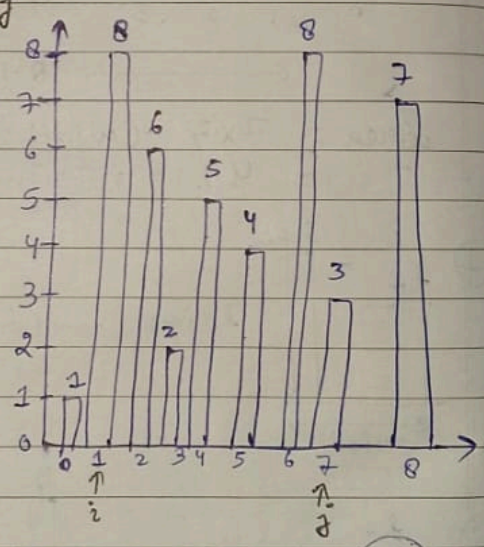


$h = 1$

$\Rightarrow w = 8 - 0 = 8$

Area = 8





ignore less height so i++

~~h = 8~~

$h = \min(8, 7) = 7$

$w = 8 - 1 = 7$

$A = 7 \times 7 = \boxed{49}$

~~take~~ $\min(8, 7) = 7$ so j--

height, j height

greedy & height max

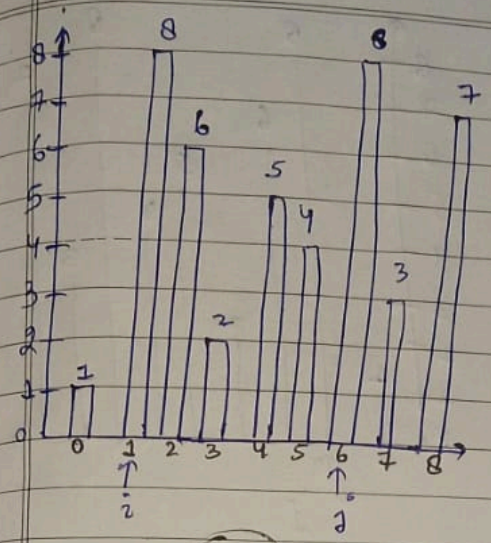
Now, $h = \min(8, 3) = 3$

$w = j - i = 7 - 1 = 6$

$A = 6 \times 3 = 18$

So not update area so, $\boxed{A = 49}$

DATE: / /
PAGE:



$h = \min(8, 8) = 8$

$w = 6 - 1 = 5$

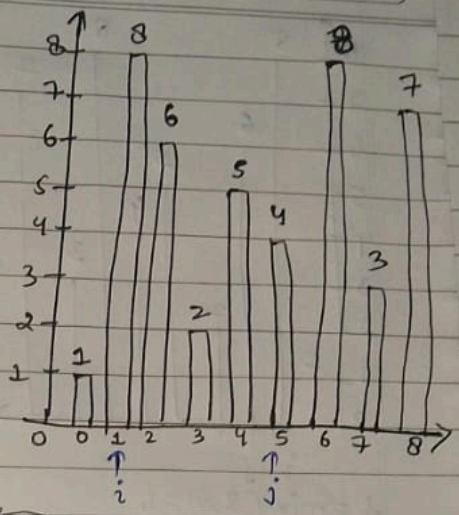
$A = 8 \times 5 = 40$

Not update area

So, $\boxed{A = 49}$

both have same height so we can move either i or j

let move j.



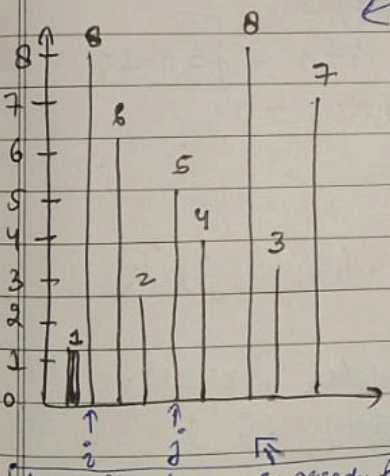
$h = \min(8, 4) = 4$

$w = 5 - 1 = 4$

$A = 4 \times 4 = 16$

Now, $h = \min(i, j)$

here j is min so move j



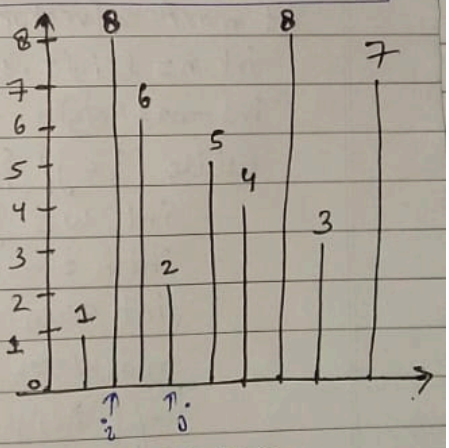
here width decrease so greedy have max height

$h = \min(8, 5) = 5$

$w = 4 - 1 = 3$

$A = 5 \times 3 = 15$

Now j is small so move j

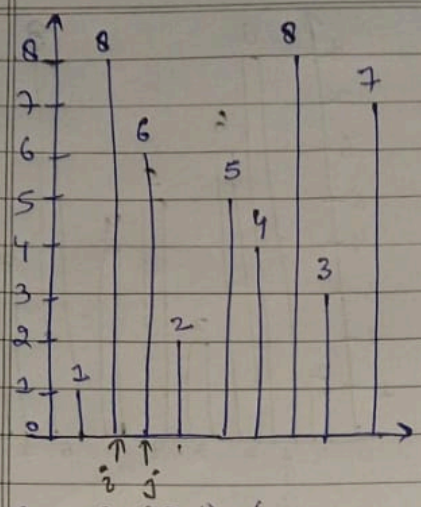


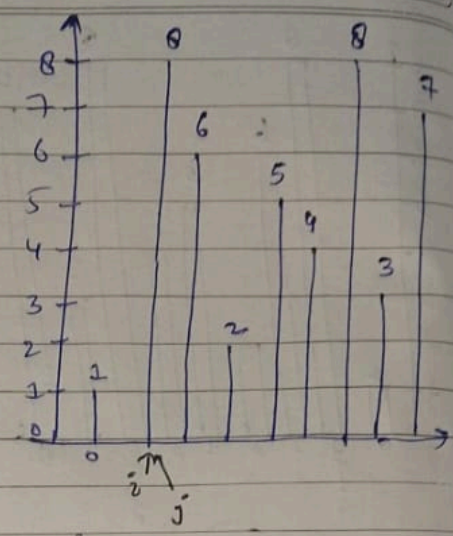
$h = \min(8, 2) = 2$

$w = 3 - 1 = 2$

$A = 2 \times 2 = 4$

DATE: / /
PAGE:





$h = \min(8, 6) = 6$

$w = 2 - 1 = 1$

$A = 6 \times 1 = \cancel{4}\ 6$

Step

⇒ **Code**

```
Class Solution {
public:
    int maxArea (vector <int> & height) {
        int n = height.size();    int i=0 , j=n-1;
        ~~int maxheight~~ .  int maxWater =0;
        while (i<j) {
            int w= j-i;
            int h = min(height[i] , height[j]);
            int area = w*h;
            maxWater = max(maxWater , area);
            if( height[i] > height[j]) {
                    j--;
            }
            else {  i++;
                }
        }
    }; return maxWater ;
```

15. 3Sum

```
class Solution {
public:
    void twoSum(vector<int>& nums, int k, vector<vector<int>>& result, int targ
et) {
        //Two pointer technique (Sorted array)

        int i = k, j = nums.size()-1;
        while(i < j) {
            if(nums[i]+nums[j] > target)
                j--;
            else if(nums[i] + nums[j] < target)
                i++;
            else {
                result.push_back({-target, nums[i], nums[j]});
                while(i < j && nums[i] == nums[i+1]) i++;
                while(i < j && nums[j] == nums[j-1]) j--;
                i++;
                j--;
            }
        }
    }
    vector<vector<int>> threeSum(vector<int>& nums) {
         if(nums.size() < 3)
            return {};
        vector<vector<int>> result;
        sort(nums.begin(), nums.end());
        for(int i = 0; i<nums.size()-2; i++) {  //(i<nums.size()-2)
            if(i!= 0 && nums[i] == nums[i-1]) {
                continue;
            }
            twoSum(nums, i+1, result, -nums[i]);
        }
        return result;
```

```
    }
};
```